

Queries to Als_20260522(五)

20260522_00

I am using a Linux notebook. How to display hidden files/directories under the VScode explorer?

In VS Code, hidden files and directories (those starting with a dot, like `.git` or `.env`) can be shown or hidden depending on your settings. Here's how you can make them visible in the Explorer on your Linux notebook:

Steps to Show Hidden Files in VS Code Explorer

1. **Open Settings**
 - Press **Ctrl + ,** (comma) or go to **File → Preferences → Settings**.
2. **Search for Files: Exclude**
 - In the search bar at the top of Settings, type:

程式碼

`files.exclude`
3. **Check Hidden File Patterns**
 - VS Code may hide dotfiles by default if `"**/*.": true` is listed under `files.exclude`.
 - If you see this entry, **uncheck it** or remove it.
4. **Search for Explorer: Exclude**
 - Similarly, check the setting `explorer.exclude`.
 - Make sure `"**/*.": true` is not enabled there.
5. **Reload Explorer**
 - After adjusting, refresh the Explorer view (click the refresh icon or reopen VS Code).
 - Hidden files and directories should now appear.

Quick Alternative

You can also directly edit your **settings.json** file:

1. Open **Command Palette** (**Ctrl + Shift + P**).
2. Type **Preferences: Open Settings (JSON)**.
3. Look for entries like:

```
json

"files.exclude": {
  "**/*.": true
}
```

Queries to Als_20260522(五)

Change `true` to `false` or remove the line entirely.

👉 Once done, you'll see hidden files (like `.gitignore`, `.env`, `.vscode`) in the Explorer.

Would you like me to also show you how to **toggle hidden files quickly** without editing settings, using the Explorer's context menu?

Queries to Als_20260522(五)

20260522_01

In the Linux notebook, DevC has finished the GitHub SSH setup: generated an SSH key pair; copy and upload the public key to GitHub. By typing “ssh -T git@github.com” in terminal, the response was “Hi xxxx! You’ve successfully authenticated, but GitHub does not provide shell access”. Does it mean that the link between remote and local repositories has been successfully connected?

- If yes, why it is required to use the command “git remote add origin url” to establish the link once more?
- If not, what is the exact meaning of “Hi xxxx! You’ve successfully authenticated, but GitHub does not provide shell access”?

Great question — this is a subtle but important distinction in Git/GitHub setup. Let’s break it down clearly:

What the SSH test means

When DevC ran:

```
bash
ssh -T git@github.com
```

and got:

程式碼

```
Hi xxxx! You've successfully authenticated, but GitHub does not provide
shell access
```

- **Success confirmation:** This proves the SSH key is correctly installed locally, uploaded to GitHub, and GitHub recognizes it.
- **No shell access:** GitHub is not a server you can log into interactively. SSH here is only for Git operations (clone, push, pull), not for opening a shell session.

👉 So yes, the authentication link between DevC’s local machine and GitHub is working.

Why `git remote add origin <url>` is still needed

- The SSH test only checks authentication.
- To actually connect a **specific local repository** to a **specific remote repository**, you must configure the remote URL.
- That’s what `git remote add origin`
`git@github.com:username/li_jing_hostel.git` does:

Queries to Als_20260522(五)

- It tells Git: “*Whenever I push or pull, use this remote repo as the source/destination.*”
- Without this, Git knows you can authenticate, but doesn’t know **which repo** to talk to.



Summary

- The SSH test proves the **identity link** (local key ↔ GitHub account).
- `git remote add origin` establishes the **repository link** (local repo ↔ remote repo).
- Both are required: one for authentication, one for repository mapping.

👉 Next step for DevC: inside the local `li_jing_hostel` project folder, run:

`bash`

```
git remote add origin git@github.com:DevA/li_jing_hostel.git
git branch -M main
git push -u origin main
```

That will fully connect DevC’s local repo to the shared GitHub project.

Would you like me to draft a **step-by-step onboarding script for DevB and DevC** (including clone, remote setup, virtualenv, Django install), so you can hand it to them as a ready-to-follow guide?

Queries to Als_20260522(五)

20260522_02

Assume that DevC created a set of SSH keys (say KEYS1) for a local repository (say REP1) in Git. DevC also created a remote repository (say REMREP1) in GitHub. By using the command “git remote add origin url”, DevC successfully establish the link between REP1 and REMREP1.

DevC then creates another local repository (say REP2) and remote repository (say REMREP2). Can DevC use KEYS1 to establish the link between REP2 and REMREP2? Please explain.

Excellent scenario — let’s clarify how SSH keys and repository links work:

One SSH key, many repositories

- **SSH keys authenticate the *user*, not the repository.**
 - KEYS1 belongs to DevC’s GitHub account.
 - Once GitHub recognizes KEYS1, DevC can use it to access *any* repository under that account (or any repo where DevC is added as a collaborator).
- That means DevC does **not** need to generate a new SSH key for REP2/REMREP2. KEYS1 will work fine.

Why `git remote add origin` is still required

- Each local repository (REP1, REP2, ...) is independent.
- Even though KEYS1 authenticates DevC to GitHub, Git still needs to know *which remote repo* each local repo should connect to.
- That’s why DevC must run:

```
bash
git remote add origin git@github.com:DevC/REMREP2.git
```

inside REP2.

- This step doesn’t create a new key — it just tells Git: “*For this local repo, use REMREP2 as the remote.*”

Summary

- **Yes, KEYS1 can be reused** for REP2 ↔ REMREP2.
- SSH keys = identity proof.
- `git remote add origin` = repository mapping.

Queries to Als_20260522(五)

- Together: KEYS1 proves DevC is allowed to access GitHub, and the remote URL tells Git which repo to talk to.

👉 Best practice: one SSH key per GitHub account is usually enough. Multiple keys are only needed if DevC manages **different GitHub accounts** (e.g., personal vs. work).

Would you like me to show you how DevC could **check which key GitHub is using** (via `ssh -vT git@github.com`), so you can confirm KEYS1 is active before pushing REP2?

Queries to Als_20260522(五)

20260522_03

What are the major differences between the two expressions followed:

- `https://github.com/`
- `git@github.com:`

Great distinction to ask about — those two expressions look similar but serve different purposes:



`https://github.com/`

- **Protocol:** Uses **HTTPS** (Hypertext Transfer Protocol Secure).
- **Usage:**
 - This is the standard web URL — you open it in a browser.
 - Git can also use HTTPS for repository operations (clone, push, pull).
- **Authentication:**
 - Typically requires username + password or a **personal access token (PAT)** when pushing code.
 - Less convenient for frequent use, since you may need to re-enter credentials unless you cache them.



`git@github.com:`

- **Protocol:** Uses **SSH** (Secure Shell).
- **Usage:**
 - This is the SSH form of the Git remote URL.
 - Example: `git@github.com:DevC/REMREP1.git`
 - You don't open this in a browser — it's for Git commands only.
- **Authentication:**
 - Relies on your SSH key pair (private key on your machine, public key uploaded to GitHub).
 - Once set up, it allows password-less authentication for Git operations.
 - More convenient for developers who push/pull often.



Key Differences

Expression	Protocol	Typical Use	Authentication
<code>https://github.com/</code>	HTTPS	Browser access, Git clone/push via HTTPS	Username + password or PAT
<code>git@github.com:</code>	SSH	Git remote operations	SSH key pair

Queries to Als_20260522(五)

👉 In practice:

- **HTTPS** is simpler if you don't want to manage SSH keys.
- **SSH** is preferred for developers — faster, more secure, and avoids repeated credential prompts.

Would you like me to show you how to **switch a repo's remote URL from HTTPS to SSH** (so DevB and DevC can standardize on SSH for easier collaboration)?